

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение  
высшего образования «Самарский национальный исследовательский  
университет имени академика С.П. Королева»  
(Самарский университет)

Институт информатики и кибернетики  
Кафедра информационных систем и технологий

ОТЧЕТ ПО ПРАКТИКЕ

Вид практики производственная  
(учебная, производственная)

Тип практики научно-исследовательская работа  
(в соответствии с ОПОП ВО)

Сроки прохождения практики: с 02.03.2026 по 06.05.2026  
(в соответствии с календарным учебным графиком)

по направлению подготовки 09.03.01 Информатика и вычислительная техника  
(уровень бакалавриата)

направленность (профиль) «Информационные системы»

Студент группы № 6404-090301D \_\_\_\_\_ М.П. Федулов

Руководитель практики,

доцент кафедры ИСТ, к.т.н. \_\_\_\_\_ Я.В. Соловьева

Дата сдачи 06.05.2026

Дата защиты 06.05.2026

Оценка \_\_\_\_\_

Самара 2026

## СОДЕРЖАНИЕ

Задание(я) для выполнения определенных видов работ, связанных с будущей профессиональной деятельностью (сбор и анализ данных и материалов, проведение исследований)

Введение

1 Обоснование выбора исходных данных. Описание тестов для проведения экспериментальных исследований по проверке эффективности разработанного программного обеспечения

2 Анализ результатов и выводы по результатам экспериментальных исследований

Заключение

Список использованных источников

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение  
высшего образования «Самарский национальный исследовательский  
университет имени академика С.П. Королева»  
(Самарский университет)

Институт информатики и кибернетики  
Кафедра информационных систем и технологий

Задание(я) для выполнения определенных видов работ, связанных с будущей  
профессиональной деятельностью (сбор и анализ данных и материалов,  
проведение исследований)

Студенту группы 6404-090301D

Федулову Михаилу Павловичу

Направление на практику оформлено приказом по университету  
от 26.02.26 г. № 121-ПР

на \_\_\_\_\_ кафедру информационных систем и технологий  
(наименование профильной организации или структурного подразделения университета)

Тема НИР: \_\_\_\_\_ «Автоматизированная система по отслеживанию выполнения  
задач в компании»

| Планируемые результаты освоения образовательной программы (компетенции)                                                                                                                                         | Планируемые результаты практики                                                                                                                                                                                           | Содержание задания                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ПК-1 Способен осуществлять проведение научно-исследовательских и опытно-конструкторских разработок по отдельным разделам темы<br>ПК 1.2 Выполняет эксперименты и оформляет результаты исследований и разработок | Знать: методологию проведения экспериментов.<br>Уметь: обосновывать выбор исходных данных для проведения экспериментов.<br>Владеть: навыками проведения экспериментов и оформления результатов исследований и разработок. | Сделать обоснование выбора исходных данных.<br><br>Провести экспериментальные исследования по проверке эффективности разработанного программного обеспечения для информационной системы по отслеживанию выполнения задач в компании.<br><br>Провести анализ результатов. |

|                                                                                                                                                                            |                                                                                                                                                                                                          |                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ПК-2 Способен осуществлять разработку тестовых случаев, проведение тестирования и исследование результатов ПК-2.2. Проводит тестирование по разработанным тестовым случаям | Знать: методы тестирования и анализа результатов тестов.<br>Уметь: обосновывать выбор тестов для тестирования программного обеспечения.<br>Владеть: навыками разработки тестов и анализа их результатов. | Разработать тесты для проведения экспериментальных исследований по проверке эффективности разработанного программного обеспечения для информационной системы по отслеживанию выполнения задач в компании.<br><br>Сделать описание тестов. |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Дата выдачи задания 02.03.2026.

Срок представления на кафедру отчета по практике 06.05.2026

Руководитель НИР,

доцент кафедры ИСТ, к.т.н. \_\_\_\_\_ Я.В. Соловьева  
(подпись)

Задание принял к исполнению

студент группы № 6404-090301D \_\_\_\_\_ М.П. Федулов  
(подпись)

## ВВЕДЕНИЕ

Реализованная система построена на двухуровневой клиент-серверной архитектуре: серверная часть разработана на базе фреймворка Spring (Java), обеспечивающего строгую типизацию, инверсию управления и встроенные механизмы безопасности, клиентская — с использованием компонентного фреймворка Angular. Приложение реализует функционал приложения для отслеживания выполнения задач, обеспечивает взаимодействие с реляционной базой данных и обработку пользовательских запросов через стандартизированный REST API.

Актуальность работы обусловлена растущими требованиями к производительности, масштабируемости и отказоустойчивости современных веб-приложений. Разделение логики на enterprise-ориентированный бэкенд на Java/Spring и строго типизированный фронтенд на Angular требует специфических подходов к тестированию, интеграции и валидации API-контрактов, существенно отличающихся от классических монолитных решений. На этапе завершения разработки ключевой задачей становится не просто реализация бизнес-функционала, а подтверждение качества, надёжности и готовности системы к эксплуатации в условиях реальной пользовательской нагрузки и конкурентного доступа к данным.

Целью работы является проведение комплексных экспериментальных исследований, направленных на верификацию бизнес-логики, оценку корректности взаимодействия компонентов системы и анализ её масштабируемости. Для достижения поставленной цели был реализован комплекс модульных и интеграционных тестов для серверной и клиентской частей; проведено имитационное моделирование поведения пользователей под нагрузкой; выработаны практические рекомендации по оптимизации архитектуры и повышению отказоустойчивости.

В ходе исследования использовались современные средства автоматизированного тестирования. Для серверной части на Spring применялись JUnit 5, Mockito и Spring Test Context для проверки контроллеров,

сервисного слоя, репозиториях и транзакционной целостности данных. Для клиентской части задействованы Jasmine/Karma в сочетании с Angular Testing Utilities для верификации компонентов, сервисов и обработки состояний UI.

Таким образом, результаты работы позволяют оценить корректность функционирования бизнес-логики, выявить узкие места в архитектуре взаимодействия Spring-бэкенда и Angular-фронтенда, а также определить степень готовности системы к промышленной эксплуатации в реальных условиях.

1 Обоснование выбора исходных данных. Описание тестов для проведения экспериментальных исследований по проверке эффективности разработанного программного обеспечения

### 1.1 Классификация видов тестирования ПО

Тестирование программного обеспечения — это стратегически важный процесс, который определяет надежность и долговечность продукта. В индустрии разработки ПО существует множество методологий и подходов к тестированию, которые могут быть классифицированы по различным критериям.

Тестирование бывает разных масштабов. Начиная от проверки отдельных компонентов (модульное или unit-тестирование) и заканчивая всеобъемлющим анализом системы (системное тестирование), каждый уровень имеет свою значимость и цели. [1]

Выделяют две крупные категории тестирования – функциональное и нефункциональное. Далее идет разделение по степени автоматизации: ручное, автоматизированное, полуавтоматизированное.

#### 1.1.1 Функциональное тестирование

Функциональное тестирование — фундаментальный процесс в обеспечении качества ПО, направленный на проверку соответствия программного продукта функциональным требованиям. По сути, это ответ на вопрос: «Делает ли система то, что должна делать?»

Основная цель функционального тестирования — убедиться, что каждая функция работает согласно спецификации. Это включает проверку пользовательского интерфейса, API, баз данных, безопасности и других аспектов системы.

#### 1.1.2 Нефункциональное тестирование

Нефункциональное тестирование выходит за рамки проверки конкретных функций и сосредотачивается на аспектах, определяющих качество взаимодействия пользователя с системой. Если функциональное тестирование

отвечает на вопрос «что делает система?», то нефункциональное — «как хорошо она это делает?».

К основным видам данного тестирования относят тестирование безопасности, производительности, надежности, совместимости, локализации.

Нефункциональные требования часто не получают должного внимания на ранних этапах разработки, однако их невыполнение может привести к серьезным проблемам. Например, система может корректно выполнять все функции, но быть настолько медленной или небезопасной, что пользователи просто откажутся с ней работать.

Процесс нефункционального тестирования требует специализированных инструментов и методик. Например, для тестирования производительности используются такие инструменты как JMeter, LoadRunner или Gatling, а для тестирования безопасности — OWASP ZAP, Burp Suite или Acunetix.

## 1.2 Модульное, интеграционное и нагрузочное тестирование

В работе используется многоуровневый подход, соответствующий концепции «пирамиды тестирования». Схема представлена на рисунке 1.



Рисунок 1 – Пирамида тестирования ПО

Модульные тесты проверяют минимальные атомарные единицы кода: методы сервисов, утилитарные функции, валидаторы. Они выполняются



быстро (миллисекунды) и позволяют локализовать ошибку с точностью до строки.

Пример: тест логики перехода статуса задачи (например, статус IN\_PROGRESS переходит в DONE). Если метод возвращает некорректное состояние или нарушает бизнес-правила, ошибка локализуется в одном сервисе, а не во всём приложении.

Интеграционные тесты проверяют взаимодействие между слоями системы. Например, корректность сохранения задачи через Spring Data JPA в PostgreSQL, работу транзакций, обработку @Valid-аннотаций в DTO, а также взаимодействие Angular-сервисов с REST-эндпоинтами. Даже при корректной работе изолированных модулей, их интеграция может вызывать сбои (разные форматы дат, несовместимые сериализаторы, нарушение контрактов API).

Системное тестирование проверяет приложение целиком. Нагрузочное – частный случай, где система испытывается при пиковых нагрузках.

### 1.3 Обоснование выбора исходных данных

Выбор исходных данных для тестирования базируется на принципах граничного тестирования, анализа классов эквивалентности и методов комбинаторного тестирования. В рамках данной работы использовались как позитивные тестовые сценарии, соответствующие корректной работе системы, так и негативные сценарии, имитирующие ошибочные действия пользователя или пограничные состояния системы. Также применялся метод мутационного тестирования для проверки устойчивости тестов к изменениям в коде.

### 1.4 Инструментарий тестирования

Для бекенда на Spring были использованы следующие инструменты:

- Junit 5: тестовый фреймворк для структурирования и выполнения тестов;
- Mockito: мокирование для изоляции зависимостей;
- Spring Boot Test: интеграционное тестирование для загрузки контекста и тестирования эндпоинтов;
- AssertJ: для проверки результатов через определенные утверждения.

Для фронтенда на Angular были использованы:

- Karma: тестовый раннер для выполнения тестов в реальных браузерах;
- Jasmine: тестовый фреймворк с интеграцией с Angular CLI;
- Angular TestBed: изоляция зависимостей и создания тестовых модулей.

Данный стек технологий позволяет наиболее эффективно и информативно провести тестирование всего приложения и его функций, в частности.

## 2 Анализ результатов и выводы по результатам экспериментальных исследований

### 2.1 Анализ результатов тестирования фронтенда

Для тестирования использовался Jasmine – фреймворк для тестирования JavaScript-кода. Он не зависит от других фреймворков и работает в браузерах и Node.js. [2] Для запуска тестирования уже используется Karma – тестовый раннер, который работает в связке с Jasmine. [3]

#### 2.1.1 Аутентификация пользователя

Тест на успешную авторизацию проверяет процесс входа пользователя в систему. Сначала подготавливаются мок-данные: имя пользователя и пароль. Затем вызывается метод `login` сервиса. С помощью `HttpTestingController` перехватывается исходящий HTTP-запрос. Проверяется, что запрос отправлен на правильный URL-адрес, имеет метод POST и корректный заголовок `Content-Type` со значением `application/x-www-form-urlencoded`. После этого имитируется успешный ответ от сервера. В конце проверяется, что состояние аутентификации в `BehaviorSubject` изменилось на `true` и сработал редирект на страницу со списком задач через `Router.navigate`.

Тест на обработку ошибки 401 проверяет реакцию приложения на неверные учетные данные. Создается шпион для функции `window.alert`. Вызывается метод `login` с заведомо неверными данными. Перехваченному запросу возвращается ошибка со статусом 401. Тест подтверждает, что пользователю было показано всплывающее окно с текстом о неверном логине или пароле и что ошибка была корректно проброшена дальше в поток.

#### 2.1.2 Получение списка задач

Тест получения списка задач проверяет метод `getTasks`. Создается массив объектов, соответствующих интерфейсу `Task` с полями `id`, `title`, `status`, `createdBy` и `createdAt`. Тест подписывается на результат метода и сверяет полученный массив с ожидаемым. `HttpTestingController` проверяет, что в URL запроса правильно подставился параметр `userId`. Это гарантирует, что фронтенд запрашивает задачи именно для текущего пользователя.

Тест создания новой задачи проверяет метод `createTask`. Он имитирует отправку объекта задачи без ID. Тест проверяет, что в теле POST-запроса передаются все нужные поля. Серверу посылается команда вернуть этот же объект, но уже с присвоенным ID и датой создания. Тест завершается успешно, если возвращенные данные совпадают с ответом сервера.

### 2.1.3 Редактирование задачи

Тест определения режима работы проверяет логику инициализации компонента. Используется мок объекта `ActivatedRoute`, в котором имитируется путь URL. Если в пути присутствует слово `new`, переменная `isCreateMode` устанавливается в `true`, и форма остается пустой. Если в пути указан ID, компонент переходит в режим редактирования, вызывает метод `getTask` у `TaskService` и заполняет поля формы полученными данными через `patchValue`. Это критически важно для корректного отображения интерфейса пользователю.

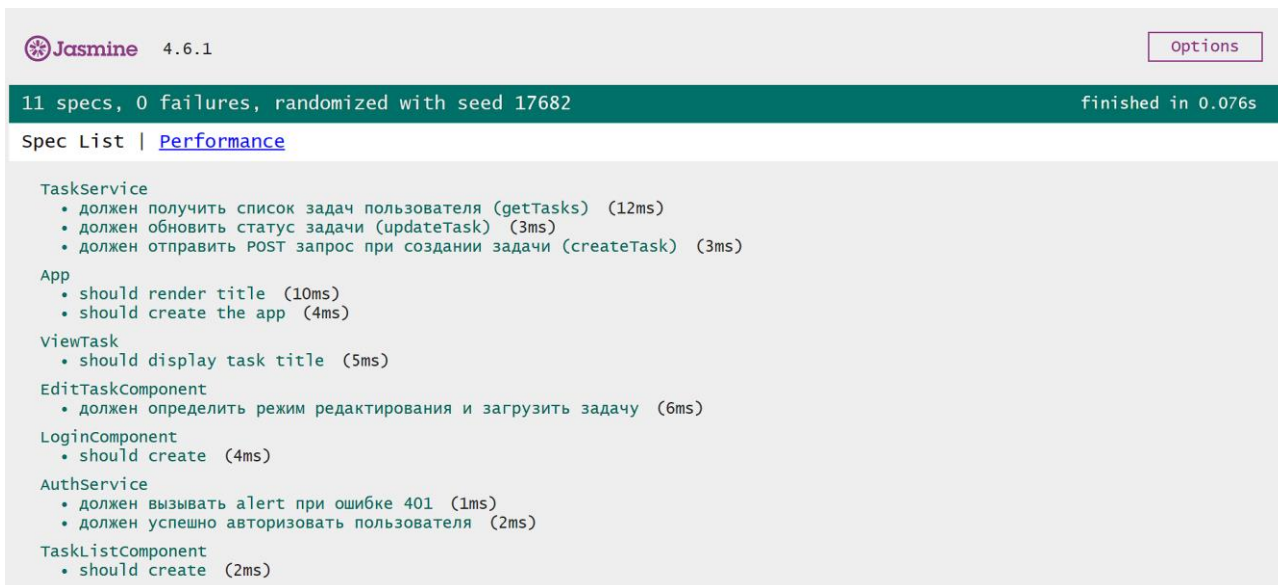
Тест отправки формы проверяет метод `onSubmit`. Он проверяет валидность формы перед выполнением действий. В режиме создания вызывается соответствующий метод сервиса, а после успешного ответа происходит навигация обратно к списку задач. Тест также проверяет обработку ошибок, например, когда бэкенд возвращает ошибку из-за превышения лимита активных задач, удостоверяясь, что пользователь видит соответствующее предупреждение.

### 2.1.3 Отображение задач

Тест отображения данных проверяет корректный рендеринг входных данных `Input`. Поскольку компонент помечен как обязательный к заполнению, тест сначала присваивает объект задачи свойству компонента и только потом запускает механизм обнаружения изменений. Проверяется, что заголовок задачи отобразился в теге `h3`, а статус прошел через механизм `switch` и отобразился на русском языке.

Тест проверки ролей проверяет логику доступа. Метод `hasRole` обращается к `sessionStorage` для получения текущей роли пользователя. Тест имитирует наличие роли `ROLE_ADMIN` и проверяет, отображается ли кнопка

удаления задачи в DOM-дереве. Это гарантирует, что рядовые пользователи не смогут видеть или нажимать на кнопки управления, предназначенные для администраторов.



```
Jasmine 4.6.1 Options
11 specs, 0 failures, randomized with seed 17682 finished in 0.076s
Spec List | Performance

TaskService
  • должен получить список задач пользователя (getTasks) (12ms)
  • должен обновить статус задачи (updateTask) (3ms)
  • должен отправить POST запрос при создании задачи (createTask) (3ms)
App
  • should render title (10ms)
  • should create the app (4ms)
ViewTask
  • should display task title (5ms)
EditTaskComponent
  • должен определить режим редактирования и загрузить задачу (6ms)
LoginComponent
  • should create (4ms)
AuthService
  • должен вызывать alert при ошибке 401 (1ms)
  • должен успешно авторизовать пользователя (2ms)
TaskListComponent
  • should create (2ms)
```

Рисунок 2 – Результаты тестирования

Как видно из результатов, тесты выполняются успешно и проверяются успешные и неудачные сценарии. Небольшое время тестирования показывает достаточно большую эффективность реализации данной части приложения.

## 2.2 Анализ тестирования бекенда

Объектами тестирования выступили следующие компоненты системы: контроллер аутентификации `AuthController`, контроллер задач `TaskController`, контроллер пользователей `UserController`, сервис управления задачами `TaskService`, а также сервис пользовательских данных `CustomUserDetailsService`. Для каждого компонента были разработаны как модульные тесты с использованием фреймворка `Mockito`, так и интеграционные тесты с применением `Spring Boot Test`.

Для оценки эффективности тестирования используется тестовое покрытие – это метрика в тестировании программного обеспечения, которая показывает, какая часть кода, функциональности или требований проверена тестами. По сути, это индикатор полноты вашего тестирования, который отвечает на вопрос: «Насколько тщательно мы протестировали продукт?». [4]

### 2.2.1 Сервис по работе с задачами

Данный набор тестов включает 10 тестовых методов, направленных на проверку бизнес-логики TaskService. Тесты выполняются изолированно от базы данных и других компонентов через мокирование зависимостей.

Тестовый метод `createTask_Success` проверяет сценарий успешного создания задачи при отсутствии активных задач у пользователя. В ходе теста имитируется пользователь с идентификатором 1, у которого счетчик активных задач возвращает значение 0. Метод `verify` проверяет, что метод `save` репозитория был вызван ровно один раз. Результат теста считается успешным, если возвращенный объект `TaskDto` не равен `null` и его заголовок соответствует переданному.

Тестовый метод `createTask_UserNotFound_ReturnsNull` моделирует ситуацию, когда в системе отсутствует пользователь с указанным именем. Репозиторий пользователей настроен на возврат `null` при поиске. Ожидаемый результат - метод сервиса возвращает `null`, а метод `save` репозитория никогда не вызывается. Данный тест валидирует корректную обработку ситуации аутентификации несуществующего пользователя.

Тестовый метод `createTask_MaxActiveTasks_ReturnsNull` проверяет соблюдение бизнес-правила, ограничивающего количество активных задач до трех. Репозиторий задач настроен на возврат значения 3 при подсчете активных задач пользователя, что соответствует достижению лимита. При попытке создания новой задачи с активным статусом метод сервиса должен вернуть `null`. Эффективность данного теста заключается в предотвращении перегрузки пользователя задачами.

Тестовый метод `updateTask_Success` проверяет корректность обновления существующей задачи. Репозиторий настроен на возврат существующей задачи при поиске по идентификатору. Входной объект `TaskDto` содержит обновленный заголовок. Ожидается, что метод не выбросит исключение, а метод `save` репозитория будет вызван для сохранения изменений.

Тестовый метод `updateTask_TaskNotFound_ThrowsException` проверяет реакцию системы на попытку обновления несуществующей задачи. Репозиторий возвращает пустой `Optional` при поиске по идентификатору. Ожидаемый результат - выбрасывание исключения `TaskNotFoundException`. Данный тест валидирует корректную обработку ошибочных идентификаторов.

Тестовый метод `updateTask_ActivatingTaskWithMaxCount_ThrowsException` является наиболее критическим, так как проверяет сложный бизнес-сценарий перевода задачи из завершенного статуса в активный при уже достигнутом лимите активных задач. Тест имитирует задачу со статусом `DONE`, которую пытаются перевести в `IN_PROGRESS` через `DTO`. При этом счетчик активных задач пользователя настроен на максимальное значение 3. Ожидается выбрасывание исключения `MaxActiveCountTaskException`, что предотвращает нарушение бизнес-правил.

Тестовый метод `findById_Success` проверяет корректность поиска задачи по идентификатору. Репозиторий возвращает `Optional` с задачей. Результат считается успешным, если возвращенный `Optional` содержит задачу с ожидаемым заголовком.

Тестовый метод `findById_NotFound` проверяет сценарий поиска несуществующей задачи. Ожидается пустой `Optional`.

Тестовый метод `countActiveTasksByUserId_Success` валидирует передачу запроса подсчета активных задач от сервиса к репозиторию. Репозиторий настроен на возврат значения 2. Тест проверяет, что сервис возвращает именно это значение без изменений.

Тестовый метод `countStatusTasks_Success` проверяет агрегацию статистики по статусам задач. Репозиторий возвращает коллекцию объектов `TaskStatusDto` с количеством задач по каждому статусу. Сервис преобразует эту коллекцию в `Map`. Тест проверяет корректность преобразования: размер карты должен соответствовать количеству статусов, а значения должны совпадать с переданными.

Тестовое покрытие данного сервиса представлено на рисунке 3.

| Элемент ^    | Класс, %   | Метод, %    | Строка, %    | Ветвь, %    |
|--------------|------------|-------------|--------------|-------------|
| ru.ssau.todo | 15% (2/13) | 27% (12/43) | 26% (38/142) | 41% (19/46) |
| controllers  | 0% (0/3)   | 0% (0/13)   | 0% (0/38)    | 0% (0/16)   |
| entity       | 0% (0/4)   | 0% (0/2)    | 0% (0/2)     | 100% (0/0)  |
| exceptions   | 50% (1/2)  | 50% (1/2)   | 50% (1/2)    | 100% (0/0)  |
| repository   | 100% (0/0) | 100% (0/0)  | 100% (0/0)   | 100% (0/0)  |
| service      | 25% (1/4)  | 42% (11/26) | 37% (37/100) | 63% (19/30) |

Рисунок 3 – Тестовое покрытие TaskService

### 2.2.2 Сервис пользовательских данных

Набор из 5 тестов проверяет функциональность аутентификации и регистрации пользователей.

Тестовый метод `loadUserByUsername_Success` проверяет успешную загрузку пользователя по имени. Репозиторий возвращает объект `User`. Тест валидирует, что результат метода `loadUserByUsername` не равен `null` и содержит корректное имя пользователя.

Тестовый метод `loadUserByUsername_UserNotFound_ThrowsException` проверяет обработку ситуации, когда запрашиваемый пользователь отсутствует в базе данных. Репозиторий возвращает `null`, что должно привести к выбрасыванию `UsernameNotFoundException`. Данный тест важен для безопасности системы, так как предотвращает разглашение информации о существовании пользователей.

Тестовый метод `registerUser_Success` проверяет полный цикл регистрации нового пользователя. Репозиторий пользователей возвращает `null` при проверке существования имени, что позволяет регистрацию. Кодировщик паролей возвращает закодированную строку. Репозиторий ролей возвращает роль `USER`. Тест проверяет, что метод регистрации завершается без исключений, а метод `save` репозитория вызывается один раз с объектом `User`, у которого установлен закодированный пароль.



Тестовый метод `registerUser_UserAlreadyExists_ThrowsException` проверяет предотвращение дублирования имен пользователей. Репозиторий возвращает существующего пользователя при проверке имени, что должно привести к выбрасыванию `UserAlreadyRegisterException`. Метод `save` репозитория при этом не должен вызываться.

Тестовый метод `registerUser_AdminUser_GetsAdminRole` проверяет автоматическое назначение административной роли при регистрации пользователя с именем `admin`. Репозиторий ролей возвращает роль `ADMIN`. Тест использует `ArgumentCaptor` для захвата сохраняемого объекта `User` и проверяет, что его коллекция ролей содержит административную роль.

Тестовое покрытие сервиса представлено на рисунке 4.

| Покрытие CustomUserDetailsServiceTest x                                                    |            |            |              |            |
|--------------------------------------------------------------------------------------------|------------|------------|--------------|------------|
| <div> <div>📁</div> <div>⬆</div> <div>⬇</div> <div>↗</div> <div>↘</div> <div>🔍</div> </div> |            |            |              |            |
| Элемент ^                                                                                  | Класс, %   | Метод, %   | Строка, %    | Ветвь, %   |
| ✓ 📁 ru.ssau.todo                                                                           | 15% (2/13) | 6% (3/43)  | 10% (15/142) | 13% (6/46) |
| > 📁 controllers                                                                            | 0% (0/3)   | 0% (0/13)  | 0% (0/38)    | 0% (0/16)  |
| > 📁 entity                                                                                 | 0% (0/4)   | 0% (0/2)   | 0% (0/2)     | 100% (0/0) |
| > 📁 exceptions                                                                             | 50% (1/2)  | 50% (1/2)  | 50% (1/2)    | 100% (0/0) |
| > 📁 repository                                                                             | 100% (0/0) | 100% (0/0) | 100% (0/0)   | 100% (0/0) |
| > 📁 service                                                                                | 25% (1/4)  | 7% (2/26)  | 14% (14/100) | 20% (6/30) |

Рисунок 4 – Тестовое покрытие CustomUserDetailsService

После проведенных тестов можно сделать вывод, что система готова к базовому использованию, но требует большего количества тестов для более детальной оценки эффективности работы.

## ЗАКЛЮЧЕНИЕ

В процессе выполнения научно-исследовательской работы были освоены индикаторы ПК-1.2, ПК-2.2 компетенций ПК-1 и ПК-2, и решены все поставленные задачи:

- было сделано обоснование выбора исходных данных;
- были разработаны тесты для проведения экспериментальных исследований по проверке эффективности разработанного программного обеспечения для информационной системы по отслеживанию выполнения задач в компании;
- было сделано описание тестов;
- были проведены экспериментальные исследования по проверке эффективности разработанного программного обеспечения информационной системы по отслеживанию выполнения задач в компании;
- был проведен анализ и сделаны выводы по результатам экспериментов.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тестирование ПО: комплексный подход к обеспечению качества систем [Электронный ресурс] // URL: <https://sky.pro/wiki/profession/vidy-testirovaniya-ot-funkcionalnogo-do-nagruzochnogo/> (дата обращения: 04.05.2026)
2. Jasmine – Simple JavaScript testing [Электронный ресурс] // URL: <https://jasmine.github.io/> (дата обращения: 04.05.2026)
3. Karma [Электронный ресурс] // URL: <https://karma-runner.github.io/2.0/intro/how-it-works.html> (дата обращения: 04.05.2026)
4. Тестовое покрытие: как измерять, улучшать и повышать качество QA [Электронный ресурс] // URL: <https://sky.pro/wiki/media/cto-takoe-testovoe-pokrytie/> (дата обращения: 04.05.2026)